MAT700 Mathematical Methods for Data Mining - Lecture5

A. Balinsky

Cardiff School of Mathematics

History

- One of the biggest changes in our lives in the decade following the turn of the century was the availability of efficient and accurate Web search, through search engines such as Google.

- While Google was not the first search engine, it was the **first able to defeat the spammers** who had made search almost useless.
  Link spammers sometimes try to make their unethicality less apparent by referring to what they do as "**search-engine optimization**".

- Moreover, the innovation provided by Google was a non-trivial technological advance, called "**PageRank**".
  The term PageRank comes from Larry Page, the inventor of the idea and a founder of Google.

- **Early Search Engines and Term Spam.**
  Techniques for fooling search engines into believing your page is about something it is not, are called *term spam*. The ability of term spammers to operate so easily rendered early search engines almost useless.

- To combat term spam, Google introduced two innovations (The PageRank algorithm was first introduced by Sergey Brin and Larry Page at the Seventh International World Wide Web Conference (WWW7) in April 1998):
  - Simulation of random surfers,
  - The content of a page was judged not only by the terms appearing on that page, but by the terms used in or near the links to that page.
- There are many other applications of the PageRank algorithm.

It is reasonable to ask why simulation of random surfers should allow us to approximate the intuitive notion of the "importance" of pages.
There are two related motivations that inspired this approach.

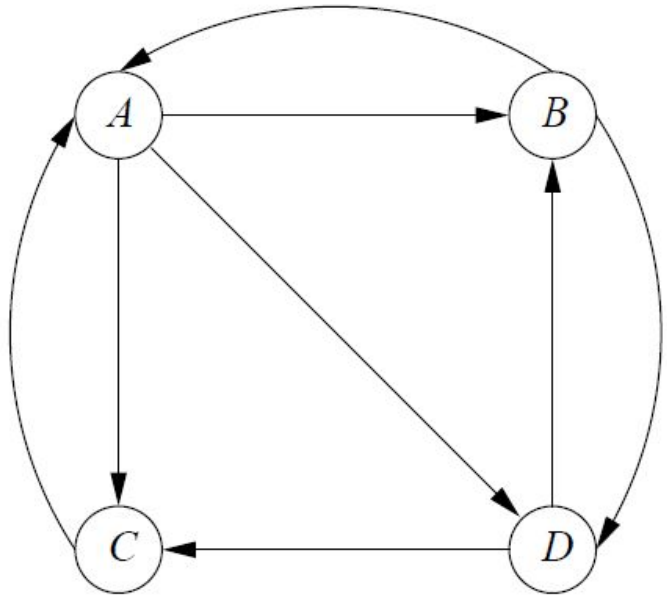
- Users of the Web "vote with their feet". They tend to place links to pages they think are good or useful pages to look at, rather than bad or useless pages.

- The behaviour of a random surfer indicates which pages users of the Web are likely to visit. Users are more likely to visit useful pages than useless pages.
  But regardless of the reason, the PageRank measure has been proved empirically to work, and so we shall study in detail how it is computed.

Definition of PageRank

- **PageRank** is a **function** that assigns a real number to each page in the Web (or at least to that portion of the Web that has been crawled and its links discovered). The intent is that the *higher* the PageRank of a page, the more *"important"* it is.
- We begin by defining the basic, idealised PageRank, and follow it by modifications that are necessary for dealing with some real-world problems concerning the structure of the Web.
- Think of the Web as a **directed graph**, where pages are the nodes, and there is an arc from page p_1 to page p_2 if there are one or more links from p_1 to p_2.

- Figure below is an example of a tiny version of the Web, where there are only four pages. Page *A* has links to each of the other three pages; page *B* has links to *A* and *D* only; page *C* has a link only to *A*, and page *D* has links to *B* and *C* only.



- Suppose a *random surfer* starts at page A in our tiny Web. There are links to B, C, and D, so this surfer will next be at each of those pages with probability 1/3, and has zero probability of being at A.
  A random surfer at B has, at the next step, probability 1/2 of being at A, 1/2 of being at D, and 0 of being at B or C.
- In general, we can define the *transition matrix of the Web* to describe what happens to random surfers after one step.

Definition (Transition Matrix of the Web)

This matrix M has n rows and columns, if there are n pages. The element m_{ij} in row i and column j has value $1/k$ if page j has k arcs out, and one of them is to page i.
Otherwise, $m_{ij} = 0$.

Example

The transition matrix for the Web of Fig. above is

$$M = \begin{bmatrix} 0 & 1/2 & 1 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

In this matrix, the order of the pages is the natural one, *A*, *B*, *C*, and *D*.

Idealized PageRank function

The probability distribution for the location of a random surfer can be described by a *column vector* whose jth component is the probability that the surfer is at page j. This probability is the (idealized) PageRank function.

Suppose we start a random surfer at any of the n pages of the Web with equal probability.

Then the initial vector v_0 will have $1/n$ for each component.

If M is the transition matrix of the Web, then after one step, the distribution of the surfer will be Mv_0 (Why?), after two steps it will be $M(Mv_0) = M^2v_0$, and so on.
In general, multiplying the initial vector v_0 by M a total of i times will give us the distribution of the surfer after i steps.

- This sort of behaviour is an example of the ancient theory of *Markov processes*.
  It is known that the distribution of the surfer approaches a limiting distribution v that satisfies $v = Mv$, provided the following condition is met:
  - The graph is **strongly connected**, that is, it is possible to get from any node to any other node.

- Note that our tiny Web satisfies this condition.
- Because M is stochastic, meaning that its columns each add up to 1, v is the **principal eigenvector** (its associated eigenvalue is the largest of all eigenvalues). Note also that, because M is stochastic, the eigenvalue associated with the principal eigenvector is 1.

- We can compute the principal eigenvector of M by starting with the initial vector v_0 and multiplying by M some number of times, until the vector we get shows little change at each round. In practice, for the Web itself, **50-75** iterations are sufficient to converge to within the error limits of double-precision arithmetic.

- You might think that the way to solve the equation $v = Mv$ is by Gaussian elimination.
  However, in realistic examples, where there are tens or hundreds of billions of nodes, Gaussian elimination is not feasible. The reason is that Gaussian elimination takes time that is cubic in the number of equations. Thus, the only way to solve equations on this scale is to iterate as we have suggested.

Example

Suppose we apply the process described above to the matrix M from our tiny web. Since there are four nodes, the initial vector v_0 has four components, each $1/4$. The sequence of approximations to the limit that we get by multiplying at each step by M is:

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} \begin{bmatrix} 9/24 \\ 5/24 \\ 5/24 \\ 5/24 \end{bmatrix} \begin{bmatrix} 15/48 \\ 11/48 \\ 11/48 \\ 11/48 \end{bmatrix} \begin{bmatrix} 11/32 \\ 7/32 \\ 7/32 \\ 7/32 \end{bmatrix} \cdots \begin{bmatrix} 3/9 \\ 2/9 \\ 2/9 \\ 2/9 \end{bmatrix}$$

Structure of the Web

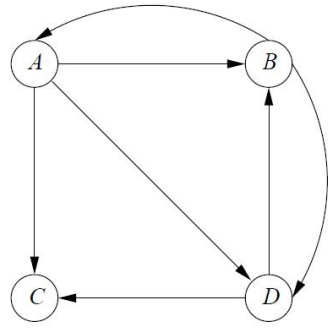
It would be nice if the Web were strongly connected.
However, it is **not**, in practice.

- As a result, PageRank is usually modified to prevent such anomalies.
- There are really **two problems** we need to avoid.
  - First is the **dead-end**, a page that has no links out. Surfers reaching such a page disappear, and the result is that in the limit no page that can reach a dead end can have any PageRank at all.
  - The second problem is groups of pages that all have outlinks but they never link to any other pages. These structures, called **spider traps**.

Avoiding Dead Ends

- Recall that a page with no link out is called a **dead end**.
- If we allow dead ends, the transition matrix of the Web is *no longer stochastic*, since some of the columns will sum to 0 rather than 1.
- A matrix whose column sums are at most 1 is called **substochastic**.
  If we compute $M^i v$ for increasing powers of a substochastic matrix M, then some or all of the components of the vector go to 0. That is, importance "drains out" of the Web, and we get no information about the relative importance of pages.

Example



C is now a dead end

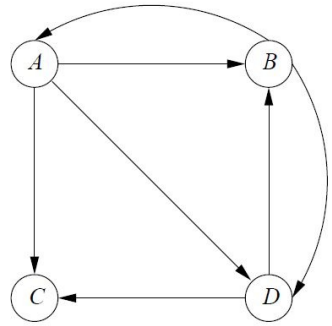
$$M = \begin{bmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} \begin{bmatrix} 3/24 \\ 5/24 \\ 5/24 \\ 5/24 \end{bmatrix} \begin{bmatrix} 5/48 \\ 7/48 \\ 7/48 \\ 7/48 \end{bmatrix} \begin{bmatrix} 21/288 \\ 31/288 \\ 31/288 \\ 31/288 \end{bmatrix} \cdots \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

- Recall that a page with no link out is called a **dead end**.
- If we allow dead ends, the transition matrix of the Web is *no longer stochastic*, since some of the columns will sum to 0 rather than 1.
- We can fix this problem by adding a *complete* set of *outgoing* links from each such dead end page i to all the pages on the Web. Thus, the transition probability of going from i to every page is 1/n, assuming a uniform probability distribution. That is, we replace each row containing all 0's with $\mathbf{e}/n$, where $\mathbf{e}$ is n-dimensional vector of all 1's.

  Below, we assume that the above is done to make M a stochastic matrix.

Example



C is now a dead end

$$M = \begin{bmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

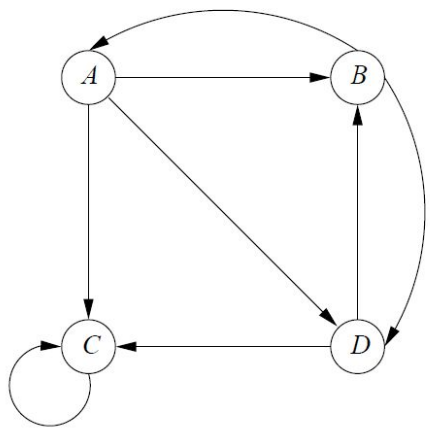
$$M = \begin{bmatrix} 0 & 1/2 & \mathbf{1/4} & 0 \\ 1/3 & 0 & \mathbf{1/4} & 1/2 \\ 1/3 & 0 & \mathbf{1/4} & 1/2 \\ 1/3 & 1/2 & \mathbf{1/4} & 0 \end{bmatrix}$$

Spider Traps and Taxation

- As we mentioned, a *spider trap* is a set of nodes with no dead ends but no arcs out. These structures can appear intentionally or unintentionally on the Web, and they cause the PageRank calculation to place all the PageRank within the spider traps.

Example

Let us consider a graph with a one-node spider trap:



- The transition matrix for this one-node spider trap is:

$$M = \begin{bmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 1 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

- If we perform the usual iteration to compute the PageRank of the nodes, we get

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} \begin{bmatrix} 3/24 \\ 5/24 \\ 11/24 \\ 5/24 \end{bmatrix} \begin{bmatrix} 5/48 \\ 7/48 \\ 29/48 \\ 7/48 \end{bmatrix} \begin{bmatrix} 21/288 \\ 31/288 \\ 205/288 \\ 31/288 \end{bmatrix} \cdots \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

As predicted, all the PageRank is at C, since a random surfer, once they reach C, can never leave.

- The transition matrix for this one-node spider trap is:

$$M = \begin{bmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 1 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

- If we perform the usual iteration to compute the PageRank of the nodes, we get

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} \begin{bmatrix} 3/24 \\ 5/24 \\ 11/24 \\ 5/24 \end{bmatrix} \begin{bmatrix} 5/48 \\ 7/48 \\ 29/48 \\ 7/48 \end{bmatrix} \begin{bmatrix} 21/288 \\ 31/288 \\ 205/288 \\ 31/288 \end{bmatrix} \cdots \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

As predicted, all the PageRank is at *C*, since a random surfer, once they reach *C*, can never leave.

- To avoid the problem of spider traps, we modify the calculation of PageRank by allowing each random surfer a small probability of **teleporting** to a random page, rather than following an out-link from their current page:

  We add a link from each page to every page and give each link a small transition probability controlled by a parameter β.

- After this augmentation, we obtain an improved PageRank model:

$$\mathbf{v} = \left((1-\beta)\frac{\mathbf{E}}{n} + \beta M \right) \mathbf{v} \tag{1}$$

  where **E** is $(ee)^T$ (**e** is a column vector of all 1's) and thus **E** is an $n \times n$ square matrix of all 1's. n is the total number of nodes in the Web graph and $1/n$ is the probability of jumping to a random page.

PageRank

- Note that Equation 1 assumes that M **has already been made a stochastic matrix**.
- The parameter β, called the *damping factor*, can be set to a value between 0 and 1. $\beta = 0.85$ is used in Google.
- Form (1) we obtain

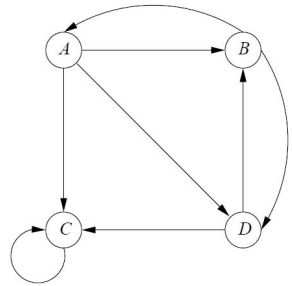
$$\mathbf{v} = (1 - \beta)\mathbf{e}/n + \beta M\mathbf{v} \tag{2}$$

- The **iterative step**, where we compute a new vector estimate of PageRanks $\mathbf{v}'$ from the current PageRank estimate $\mathbf{v}$ and the transition matrix M is

$$\mathbf{v}' = (1 - \beta)\mathbf{e}/n + \beta M\mathbf{v}$$

Example

- Let us see how the new approach to computing PageRank fares on our one-node spider trap



We shall use $\beta = 0.8$ in this example.

- The equation for the iteration becomes

$$\mathbf{v}' = \begin{bmatrix} 0 & 2/5 & 0 & 0 \\ 4/15 & 0 & 0 & 2/5 \\ 4/15 & 0 & 4/5 & 2/5 \\ 4/15 & 2/5 & 0 & 0 \end{bmatrix} \mathbf{v} + \begin{bmatrix} 1/20 \\ 1/20 \\ 1/20 \\ 1/20 \end{bmatrix}$$

Example

- Notice that we have incorporated the factor β into M by multiplying each of its elements by $4/5$. The components of the vector $(1-\beta)\mathbf{e}/n$ are each $1/20$, since $1-\beta = 1/5$ and $n = 4$.

- Here are the first few iterations:

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} \begin{bmatrix} 9/60 \\ 13/60 \\ 25/60 \\ 13/60 \end{bmatrix} \begin{bmatrix} 41/300 \\ 53/300 \\ 153/300 \\ 53/300 \end{bmatrix} \begin{bmatrix} 543/4500 \\ 707/4500 \\ 2543/4500 \\ 707/4500 \end{bmatrix} \cdots \begin{bmatrix} 15/148 \\ 19/148 \\ 95/148 \\ 19/148 \end{bmatrix}$$

- By being a spider trap, C has managed to get more than half of the PageRank for itself. However, the effect has been limited, and each of the nodes gets some of the PageRank.